Employee Attrition — Feature Engineering Pipeline

Business Problem: A multinational technology organization wants to understand the drivers of employee attrition and prepare a machine-learning-ready dataset for future predictive modelling.

Objective of this notebook: Explore the HR dataset, identify data quality issues, design and justify a feature engineering strategy, construct and select meaningful features, evaluate dimensionality reduction, and build a single reusable `scikit-learn` preprocessing pipeline.

Structure

- Part A — Understanding the Dataset
- Part B — Designing a Feature Engineering Strategy
- Part C — Feature Construction & Feature Selection
- Part D — Dimensionality Reduction & Pipeline Design
- Part E — Summary & Reflection

> **Assumption:** `Graded_Assignment_Dataset.csv` is located in the same folder as this notebook. The notebook runs top-to-bottom without manual intervention.

0. Setup

```
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder
from sklearn.compose import ColumnTransformer
from sklearn.decomposition import PCA
from sklearn.feature_selection import mutual_info_classif
from sklearn.model_selection import train_test_split

%matplotlib inline
sns.set_theme(style="whitegrid", palette="deep")
plt.rcParams['figure.dpi'] = 100
pd.set_option('display.max_columns', 50)
pd.set_option('display.width', 160)

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
```

```
df_raw = pd.read_csv('Graded Assignment Dataset.csv')
df = df_raw.copy()
print(f"Dataset shape: {df.shape[0]} rows x {df.shape[1]} columns")
df.head()
```

```
Dataset shape: 5000 rows x 39 columns
```

	Employee_ID	Age	Gender	Marital_Status	Education_Level	Department	Job_Role	Total_Work_Experience	Years_at_Company	Year
0	EMP100001	37	Male	Single	Bachelor	Engineering	Senior Engineer	15	7.8	
1	EMP100002	27	Male	Single	Bachelor	Operations	Engineer	2	2.0	
2	EMP100003	41	Male	Married	Master	Finance	Manager	15	4.7	
3	EMP100004	43	Male	Divorced	Bachelor	Finance	Manager	23	3.0	
4	EMP100005	21	Male	Single	Bachelor	HR	Associate	0	0.0	

Part A — Understanding the Dataset

A.1 Structure & Feature Types

```
info_df = pd.DataFrame({
    'dtype': df.dtypes.astype(str),
    'n_unique': df.nunique(),
    'n_missing': df.isnull().sum(),
    'pct_missing': (df.isnull().sum() / len(df) * 100).round(2)
})
info_df
```

	dtype	n_unique	n_missing	pct_missing
Employee_ID	object	5000	0	0.0
Age	int64	40	0	0.0
Gender	object	3	0	0.0
Marital_Status	object	3	0	0.0
Education_Level	object	4	0	0.0
Department	object	7	0	0.0
Job_Role	object	6	0	0.0
Total_Work_Experience	int64	42	0	0.0
Years_at_Company	float64	146	0	0.0
Years_in_Current_Role	float64	114	0	0.0
Promotion_Count	int64	5	0	0.0
Business_Travel	object	3	0	0.0
Work_Mode	object	3	0	0.0
Distance_From_Home	float64	554	0	0.0
Monthly_Salary	float64	4996	0	0.0
Annual_Bonus	float64	4698	300	6.0
Stock_Option_Value	float64	2021	0	0.0
Overtime_Hours	float64	378	0	0.0
Monthly_Expenses	float64	4990	0	0.0
Compensation_Band	object	3	0	0.0
Performance_Rating	int64	5	0	0.0
Manager_Rating	float64	5	250	5.0
Training_Hours	float64	274	350	7.0
Certification_Count	int64	8	0	0.0
Project_Count	int64	17	0	0.0
Job_Satisfaction	int64	10	0	0.0
Work_Life_Balance	int64	10	0	0.0
Environment_Satisfaction	float64	10	250	5.0
Relationship_Satisfaction	int64	10	0	0.0
Recognition_Score	int64	10	0	0.0
Manager_Support	int64	10	0	0.0
Engagement_Index	float64	33	0	0.0
Productivity_Index	float64	3204	0	0.0
Financial_Stress_Index	float64	3401	0	0.0
Attendance_Score	float64	1597	0	0.0
Career_Growth_Index	float64	2834	0	0.0
Annual_Compensation	float64	5000	0	0.0
Experience_Band	object	4	0	0.0
Attrition	int64	2	0	0.0

Next steps: Generate code with `info_df` | New interactive sheet

The dataset has **5,000 employee records and 39 columns** (38 predictors + the `Attrition` target). Based on cardinality and the data dictionary, the columns split into four groups:

Group	Columns
Identifier	`Employee_ID` (unique per row — no predictive value)
Nominal categorical	`Gender`, `Marital_Status`, `Department`, `Job_Role`, `Business_Travel`, `Work_Mode`

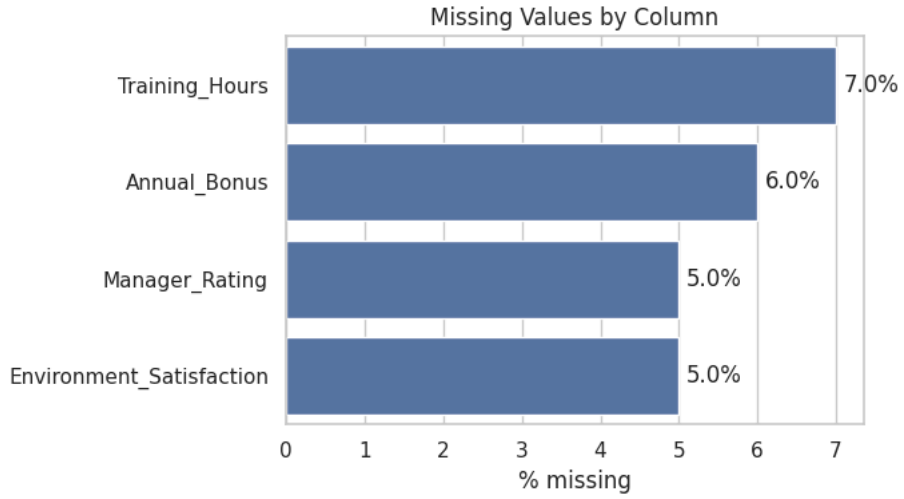
Group	Columns
Ordinal categorical	`Education_Level`, `Compensation_Band`, `Experience_Band`
Numeric (continuous / count)	The remaining 25 columns — demographic, tenure, compensation, performance, learning, and survey-based engagement measures
Target	`Attrition` (binary: 0 = Stayed, 1 = Left)

A.2 Missing Values

```
missing = info_df[info_df['n_missing'] > 0].sort_values('n_missing', ascending=False)
print(missing[['n_missing', 'pct_missing']])

fig, ax = plt.subplots(figsize=(7, 4))
sns.barplot(x=missing['pct_missing'], y=missing.index, ax=ax, color='#4C72B0')
ax.set_xlabel('% missing')
ax.set_ylabel('')
ax.set_title('Missing Values by Column')
for i, v in enumerate(missing['pct_missing']):
    ax.text(v + 0.1, i, f"{v}%", va='center')
plt.tight_layout()
plt.show()
```

```
                          n_missing  pct_missing
Training_Hours                  350          7.0
Annual_Bonus                    300          6.0
Manager_Rating                  250          5.0
Environment_Satisfaction        250          5.0
```




```
# Is missingness related to the target (i.e. informative / MNAR-like) or roughly random?
for c in missing.index:
    rate_missing = df.loc[df[c].isnull(), 'Attrition'].mean()
    print(f"{c:28s} attrition rate when missing = {rate_missing:.3f}  |  overall = {df['Attrition'].mean():.3f}")
```

```
Training_Hours               attrition rate when missing = 0.160  |  overall = 0.191
Annual_Bonus                 attrition rate when missing = 0.210  |  overall = 0.191
Manager_Rating               attrition rate when missing = 0.176  |  overall = 0.191
Environment_Satisfaction     attrition rate when missing = 0.184  |  overall = 0.191
```

Observation: Four columns have missing values — `Training_Hours` (7.0%), `Annual_Bonus` (6.0%), `Manager_Rating` (5.0%), and `Environment_Satisfaction` (5.0%). No other column has missing data, and there are no missing values in `Attrition` or the categorical columns. The attrition rate among records with a missing value is close to the overall base rate (~19%) for every affected column, which suggests the missingness is **not systematically related to the outcome** (consistent with data missing completely/randomly at the source-system level, e.g. a survey not submitted or a training record not yet logged) rather than a pattern that itself signals attrition risk. This supports using a straightforward, defensible imputation strategy rather than a more complex missing-not-at-random treatment (see Part B).

A.3 Duplicates & Target Balance

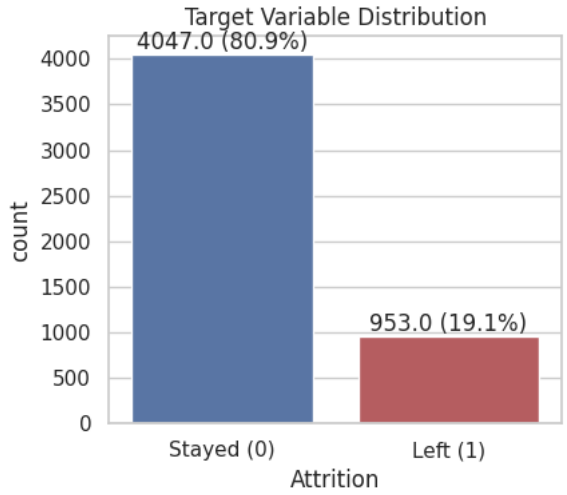
```
print("Fully duplicated rows:", df.duplicated().sum())
print("Duplicated Employee_ID:", df['Employee_ID'].duplicated().sum())
print()
target_counts = df['Attrition'].value_counts().rename({0: 'Stayed (0)', 1: 'Left (1)'})
print(target_counts)
print((target_counts / len(df) * 100).round(2).astype(str) + ' %')

fig, ax = plt.subplots(figsize=(4.5, 4))
sns.countplot(x='Attrition', data=df, ax=ax, palette=['#4C72B0', '#C44E52'])
ax.set_xticklabels(['Stayed (0)', 'Left (1)'])
```

```
ax.set_title('Target Variable Distribution')
for p in ax.patches:
    ax.annotate(f'{p.get_height()} ({p.get_height()/len(df)*100:.1f}%)',
                (p.get_x() + p.get_width()/2, p.get_height()), ha='center', va='bottom')
plt.tight_layout()
plt.show()
```

```
Fully duplicated rows: 0
Duplicated Employee_ID: 0

Attrition
Stayed (0)    4047
Left (1)       953
Name: count, dtype: int64
Attrition
Stayed (0)    80.94 %
Left (1)      19.06 %
Name: count, dtype: object
```



Observation: No fully duplicated rows and no duplicated `Employee_ID` values — the identifier is clean. The target is **imbalanced**: ~80.9% stayed vs ~19.1% left (roughly 4.2 : 1). This isn't corrected in this assignment (no model is being built), but it is worth flagging for whoever builds the predictive model downstream — stratified train/test splitting and class-imbalance-aware evaluation will matter. We use a **stratified split** later in Part D purely to demonstrate the pipeline correctly, for exactly this reason.

A.4 Numerical Distributions & Skewness

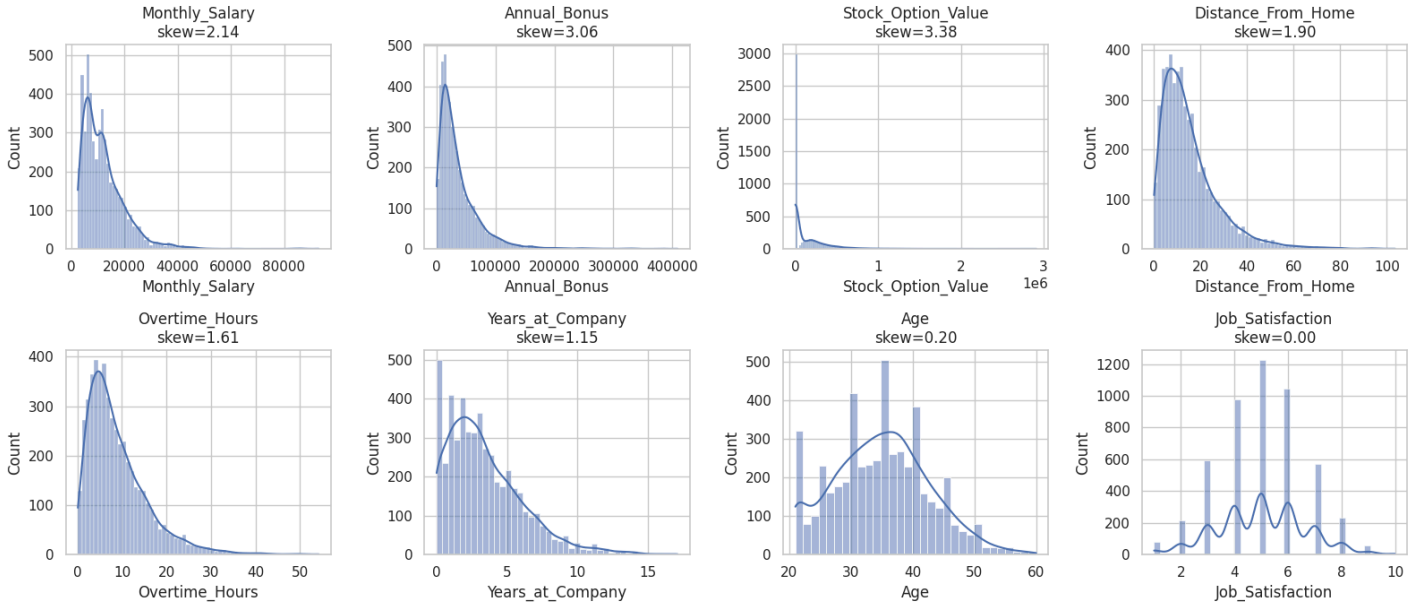
```
num_cols = df.select_dtypes(include=np.number).columns.tolist()
num_cols = [c for c in num_cols if c != 'Attrition']

skew_table = df[num_cols].skew().sort_values(ascending=False).rename('skewness').to_frame()
skew_table['interpretation'] = pd.cut(
    skew_table['skewness'].abs(),
    bins=[-0.01, 0.5, 1.0, np.inf],
    labels=['approx. symmetric', 'moderately skewed', 'highly skewed']
)
skew_table
```

	skewness	interpretation
Stock_Option_Value	3.380210	highly skewed
Annual_Bonus	3.060238	highly skewed
Annual_Compensation	2.163644	highly skewed
Monthly_Salary	2.142931	highly skewed
Years_in_Current_Role	1.923618	highly skewed
Distance_From_Home	1.895048	highly skewed
Overtime_Hours	1.611029	highly skewed
Monthly_Expenses	1.600538	highly skewed
Certification_Count	1.386466	highly skewed
Years_at_Company	1.152443	highly skewed
Promotion_Count	1.023776	highly skewed
Project_Count	0.656332	moderately skewed
Career_Growth_Index	0.398571	approx. symmetric
Total_Work_Experience	0.308548	approx. symmetric
Age	0.200330	approx. symmetric
Productivity_Index	0.057029	approx. symmetric
Relationship_Satisfaction	0.037414	approx. symmetric
Environment_Satisfaction	0.031318	approx. symmetric
Work_Life_Balance	0.022795	approx. symmetric
Engagement_Index	0.005303	approx. symmetric
Job_Satisfaction	0.003319	approx. symmetric
Recognition_Score	0.002798	approx. symmetric
Training_Hours	-0.008282	approx. symmetric
Manager_Support	-0.021726	approx. symmetric
Performance_Rating	-0.031627	approx. symmetric
Financial_Stress_Index	-0.042536	approx. symmetric
Manager_Rating	-0.088611	approx. symmetric
Attendance_Score	-0.390593	approx. symmetric

Next steps: Generate code with `skew_table` | New interactive sheet

```
plot_cols = ['Monthly_Salary', 'Annual_Bonus', 'Stock_Option_Value', 'Distance_From_Home',
             'Overtime_Hours', 'Years_at_Company', 'Age', 'Job_Satisfaction']
fig, axes = plt.subplots(2, 4, figsize=(16, 7))
for ax, c in zip(axes.flat, plot_cols):
    sns.histplot(df[c].dropna(), kde=True, ax=ax, color='#4C72B0')
    ax.set_title(f"{c}\nskew={df[c].skew():.2f}")
plt.tight_layout()
plt.show()
```



Observation: Several monetary and tenure-related variables are **strongly right-skewed** — `Stock_Option_Value` (skew ≈ 3.4, with ~60% of employees holding zero stock options — a zero-inflated variable), `Annual_Bonus` (≈ 3.1), `Monthly_Salary` (≈ 2.1), `Distance_From_Home` (≈ 1.9), `Years_in_Current_Role` (≈ 1.9), `Overtime_Hours` (≈ 1.6) and `Monthly_Expenses` (≈ 1.6). This is expected — compensation and tenure variables in real organizations are long-tailed (a small number of senior/highly-paid employees pull the distribution right). The ordinal survey scores (`Job_Satisfaction`, `Work_Life_Balance`, etc.) and demographic variables (`Age`) are close to symmetric. Skewed variables will benefit from a log-style transform before use in distance- or gradient-based models (Part B).

A.5 Outlier Detection

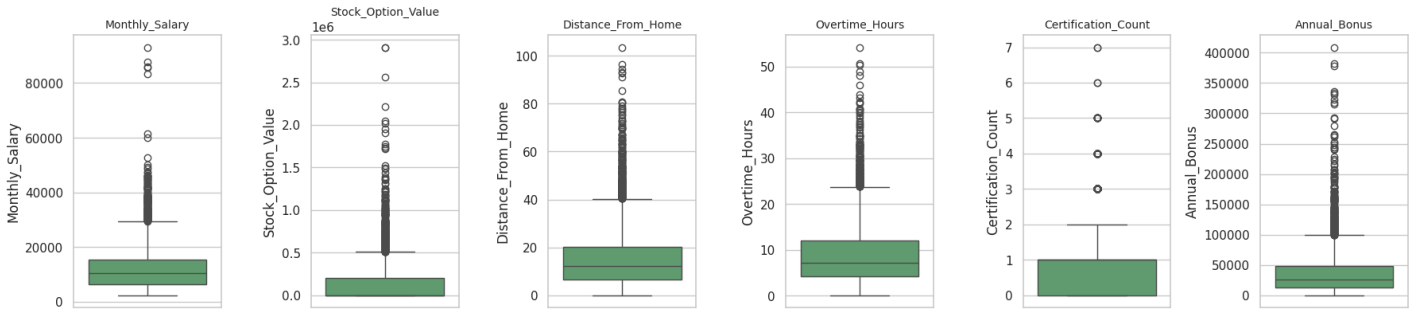
```
def iqr_outlier_summary(frame, cols, k=1.5):
    rows = []
    for c in cols:
        q1, q3 = frame[c].quantile([0.25, 0.75])
        iqr = q3 - q1
        lo, hi = q1 - k * iqr, q3 + k * iqr
        n_out = ((frame[c] < lo) | (frame[c] > hi)).sum()
        if n_out > 0:
            rows.append([c, n_out, round(n_out / len(frame) * 100, 2), round(lo, 2), round(hi, 2)])
    return pd.DataFrame(rows, columns=['feature', 'n_outliers', 'pct_outliers', 'iqr_low', 'iqr_high']) \
             .sort_values('pct_outliers', ascending=False)

outlier_summary = iqr_outlier_summary(df, num_cols)
outlier_summary.head(15)
```

	feature	n_outliers	pct_outliers	iqr_low	iqr_high
8	Stock_Option_Value	316	6.32	-305760.76	509601.26
14	Certification_Count	298	5.96	-1.50	2.50
7	Annual_Bonus	261	5.22	-38077.81	99583.76
3	Years_in_Current_Role	244	4.88	-2.60	5.40
5	Distance_From_Home	224	4.48	-13.30	40.30
9	Overtime_Hours	195	3.90	-7.54	23.76
10	Monthly_Expenses	188	3.76	-4460.63	16765.14
6	Monthly_Salary	163	3.26	-7600.54	29413.27
27	Annual_Compensation	156	3.12	-429247.93	980081.32
2	Years_at_Company	120	2.40	-3.85	10.15
12	Manager_Rating	117	2.34	1.50	5.50
4	Promotion_Count	80	1.60	-1.50	2.50
11	Performance_Rating	66	1.32	1.50	5.50
25	Attendance_Score	62	1.24	77.57	98.58
22	Engagement_Index	42	0.84	2.00	8.00

Next steps: Generate code with `outlier_summary` New interactive sheet

```
box_cols = ['Monthly_Salary', 'Stock_Option_Value', 'Distance_From_Home',
            'Overtime_Hours', 'Certification_Count', 'Annual_Bonus']
fig, axes = plt.subplots(1, 6, figsize=(18, 4))
for ax, c in zip(axes, box_cols):
    sns.boxplot(y=df[c], ax=ax, color='#55A868')
    ax.set_title(c, fontsize=10)
plt.tight_layout()
plt.show()
```

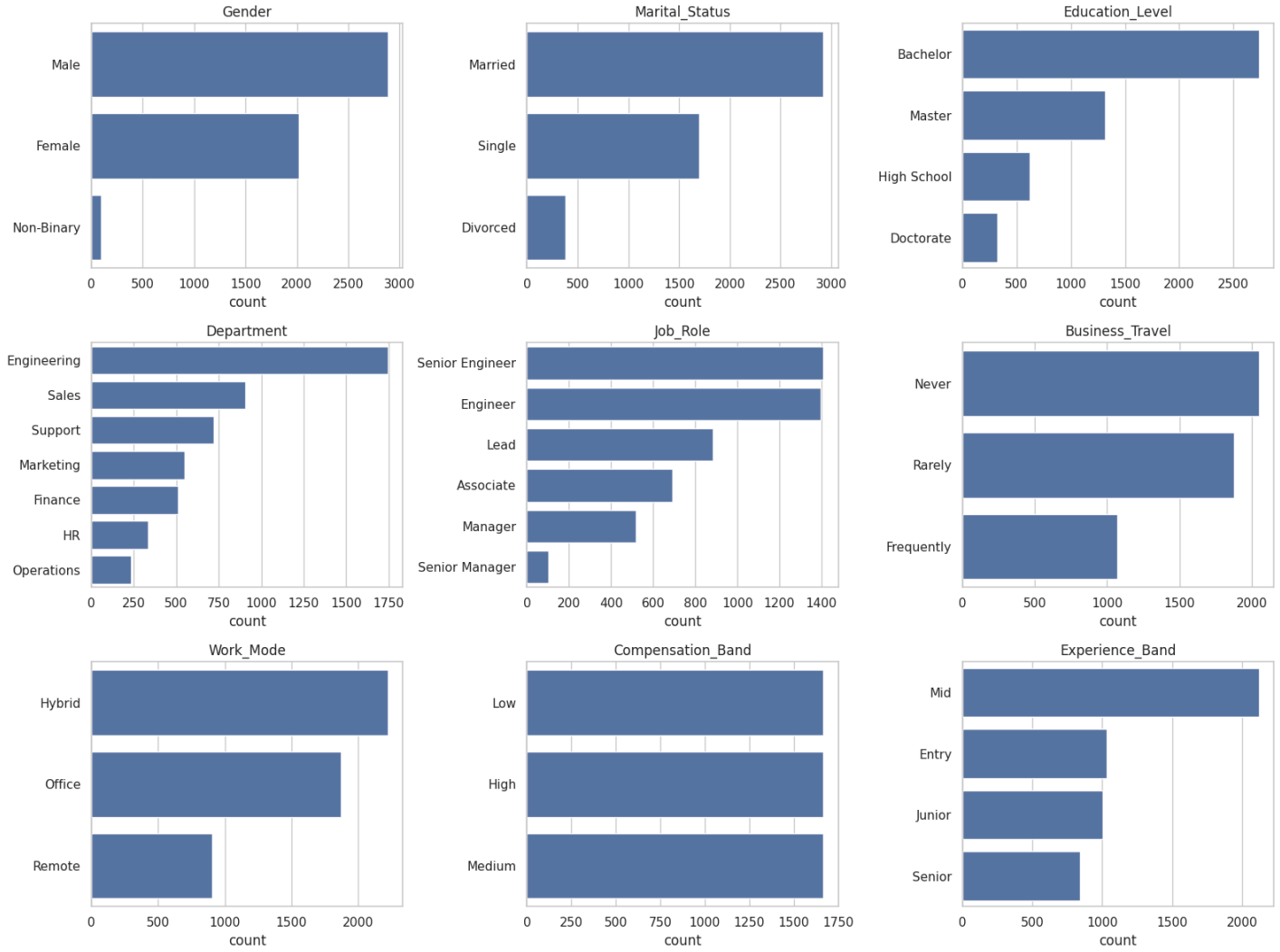


Observation: IQR-based screening flags outliers in most numeric columns (commonly 2–6% of records), concentrated in exactly the skewed monetary/tenure variables identified above (`Stock_Option_Value`, `Certification_Count`, `Annual_Bonus`, `Distance_From_Home`, `Overtime_Hours`, `Monthly_Salary`, etc.). Given the moderate volumes involved (never more than ~6% of rows) and that these values are plausible for a large multinational workforce (senior staff with large stock grants, remote employees with long commutes), we treat them as **genuine but extreme values** rather than data-entry errors — the appropriate response is **capping (winsorizing)**, not deletion, so that extreme values are retained but prevented from dominating scale-sensitive downstream models (Part B).

A.6 Categorical Variable Distributions

```
cat_cols = ['Gender', 'Marital_Status', 'Education_Level', 'Department',
            'Job_Role', 'Business_Travel', 'Work_Mode', 'Compensation_Band', 'Experience_Band']
fig, axes = plt.subplots(3, 3, figsize=(16, 12))
for ax, c in zip(axes.flat, cat_cols):
    order = df[c].value_counts().index
    sns.countplot(y=c, data=df, order=order, ax=ax, color='#4C72B0')
    ax.set_title(c)
    ax.set_ylabel('')
plt.tight_layout()
```

```
plt.show()
```

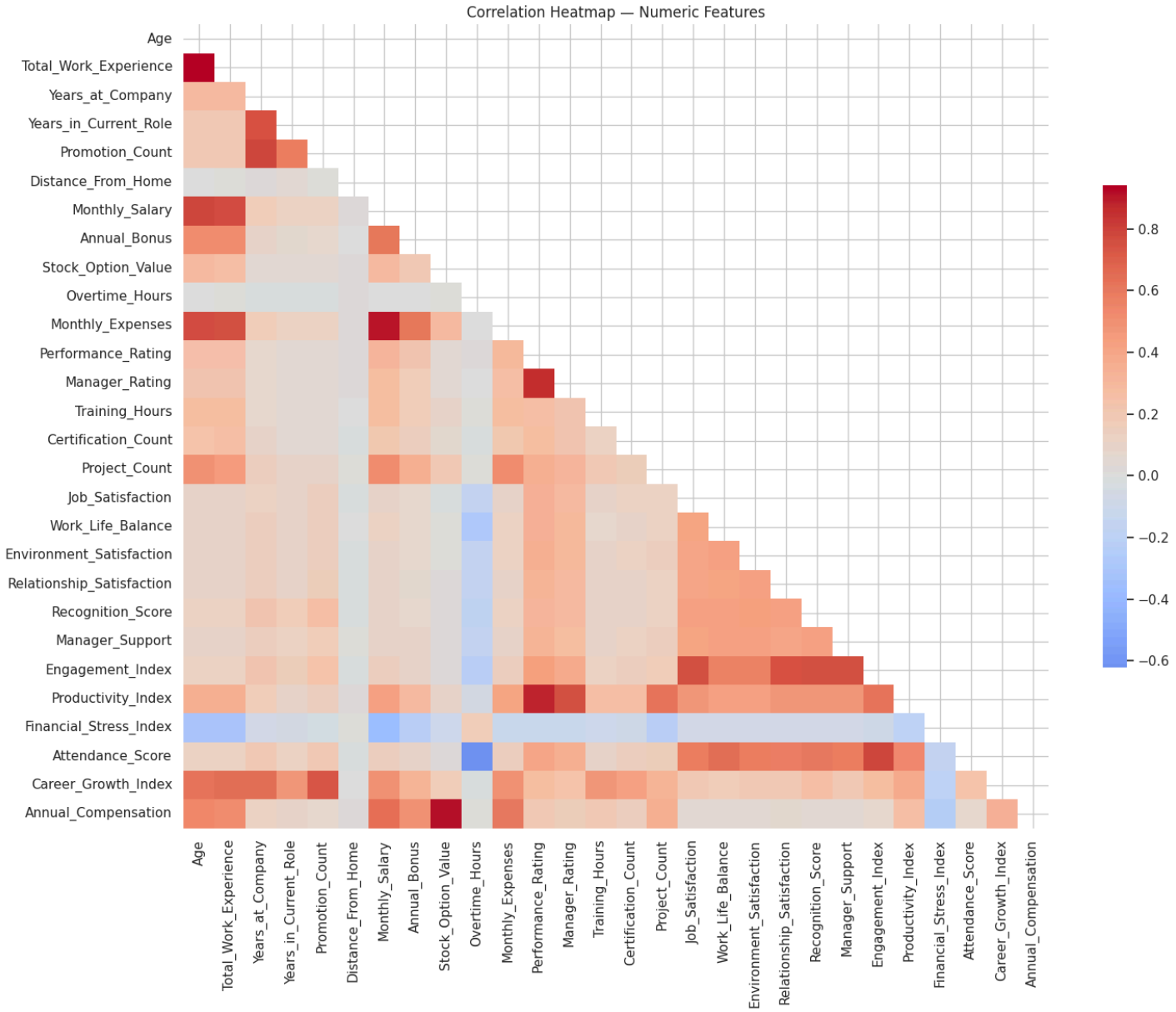


Observation: Categorical variables show realistic organizational proportions — e.g. `Department` is dominated by Engineering (~~35%~~) ~~with smaller HR/Operations groups,~~ `Job_Role` ~~skews toward Engineer/Senior Engineer, and~~ `Gender` ~~includes a small Non-Binary category~~ (1.9%) that will need `handle_unknown='ignore'`-style encoding safety for any new/rare categories at inference time. No category is so rare (<1%) that it needs to be merged into an "Other" bucket, except possibly `Gender = Non-Binary`, which we retain as-is since collapsing a demographic category is a substantive (not purely technical) decision best left to the business/ethics owner of the model.

A.7 Correlation Structure

```
plt.figure(figsize=(15, 12))
corr = df[num_cols].corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, cmap='coolwarm', center=0, annot=False, cbar_kws={'shrink': 0.6})
plt.title('Correlation Heatmap — Numeric Features')
plt.tight_layout()
```

```
plt.show()
```




```
pairs = corr.where(~mask).stack().reset_index()
pairs.columns = ['feature_1', 'feature_2', 'corr']
pairs = pairs[pairs['feature_1'] != pairs['feature_2']]
pairs['abs_corr'] = pairs['corr'].abs()
top_pairs = pairs.sort_values('abs_corr', ascending=False).head(15).drop(columns='abs_corr')
top_pairs
```

	feature_1	feature_2	corr
0	Total_Work_Experience	Age	0.941287
359	Annual_Compensation	Stock_Option_Value	0.916646
51	Monthly_Expenses	Monthly_Salary	0.909326
264	Productivity_Index	Performance_Rating	0.882006
77	Manager_Rating	Performance_Rating	0.862883
8	Promotion_Count	Years_at_Company	0.792502
322	Attendance_Score	Engagement_Index	0.785183
15	Monthly_Salary	Age	0.777244
45	Monthly_Expenses	Age	0.769544
16	Monthly_Salary	Total_Work_Experience	0.764651
251	Engagement_Index	Recognition_Score	0.760560
265	Productivity_Index	Manager_Rating	0.759929
46	Monthly_Expenses	Total_Work_Experience	0.759650
247	Engagement_Index	Job_Satisfaction	0.748640
252	Engagement_Index	Manager_Support	0.747077

Next steps: Generate code with `top_pairs` New interactive sheet

```
# Specific redundancy checks called out in the data dictionary
computed_annual_comp = df['Monthly_Salary'] * 12 + df['Annual_Bonus'].fillna(0) + df['Stock_Option_Value']
print("corr(Annual_Compensation, Salary*12+Bonus+Stock):",
      round(df['Annual_Compensation'].corr(computed_annual_comp), 4))

sat_cols = ['Job_Satisfaction', 'Work_Life_Balance', 'Environment_Satisfaction',
            'Relationship_Satisfaction', 'Recognition_Score', 'Manager_Support']
print("corr(Engagement_Index, mean of 6 satisfaction sub-scores):",
      round(df['Engagement_Index'].corr(df[sat_cols].mean(axis=1)), 4))

print()
print("Experience_Band ranges vs Total_Work_Experience:")
print(df.groupby('Experience_Band')['Total_Work_Experience'].agg(['min', 'max']))
```

```
corr(Annual_Compensation, Salary*12+Bonus+Stock): 0.9989
corr(Engagement_Index, mean of 6 satisfaction sub-scores): 0.9564

Experience_Band ranges vs Total_Work_Experience:
                 min  max
Experience_Band
Entry              0    5
Junior             6   10
Mid               11   20
Senior            21   41
```

Observation — three redundant / derived columns identified:

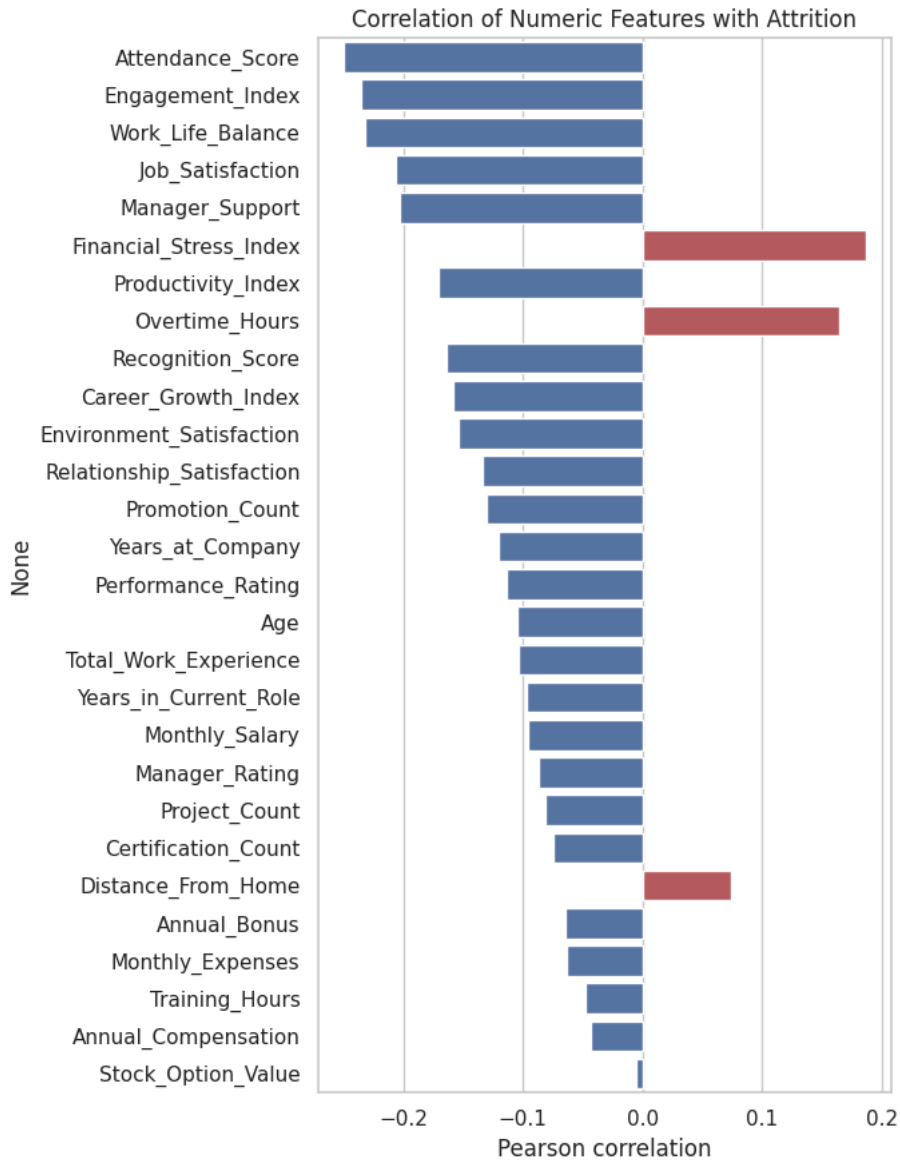
1. **`Annual_Compensation`** is (numerically, r ≈ 0.999) an exact linear combination of `Monthly_Salary`, `Annual_Bonus`, and `Stock_Option_Value` (`Monthly_Salary×12
   - Annual_Bonus + Stock_Option_Value`). Keeping all four introduces perfect multicollinearity.
2. **`Engagement_Index`** correlates at r ≈ 0.96 with the simple average of the six satisfaction/support sub-scores (`Job_Satisfaction`, `Work_Life_Balance`, `Environment_Satisfaction`, `Relationship_Satisfaction`, `Recognition_Score`, `Manager_Support`) — it is effectively a pre-computed composite of columns we already have individually, at the cost of losing granularity.
3. **`Experience_Band`** is a deterministic bucketing of `Total_Work_Experience` (Entry 0–5 / Junior 6–10 / Mid 11–20 / Senior 21+) — no new information beyond the continuous column.

Other strong pairwise correlations (`Age`–`Total_Work_Experience` r≈0.94, `Monthly_Salary`–`Monthly_Expenses` r≈0.91, `Performance_Rating`–`Productivity_Index` r≈0.88, `Performance_Rating`–`Manager_Rating` r≈0.86) are **plausible business relationships** rather than exact duplicates, so we flag them for feature-selection judgement in Part C rather than removing them outright here.

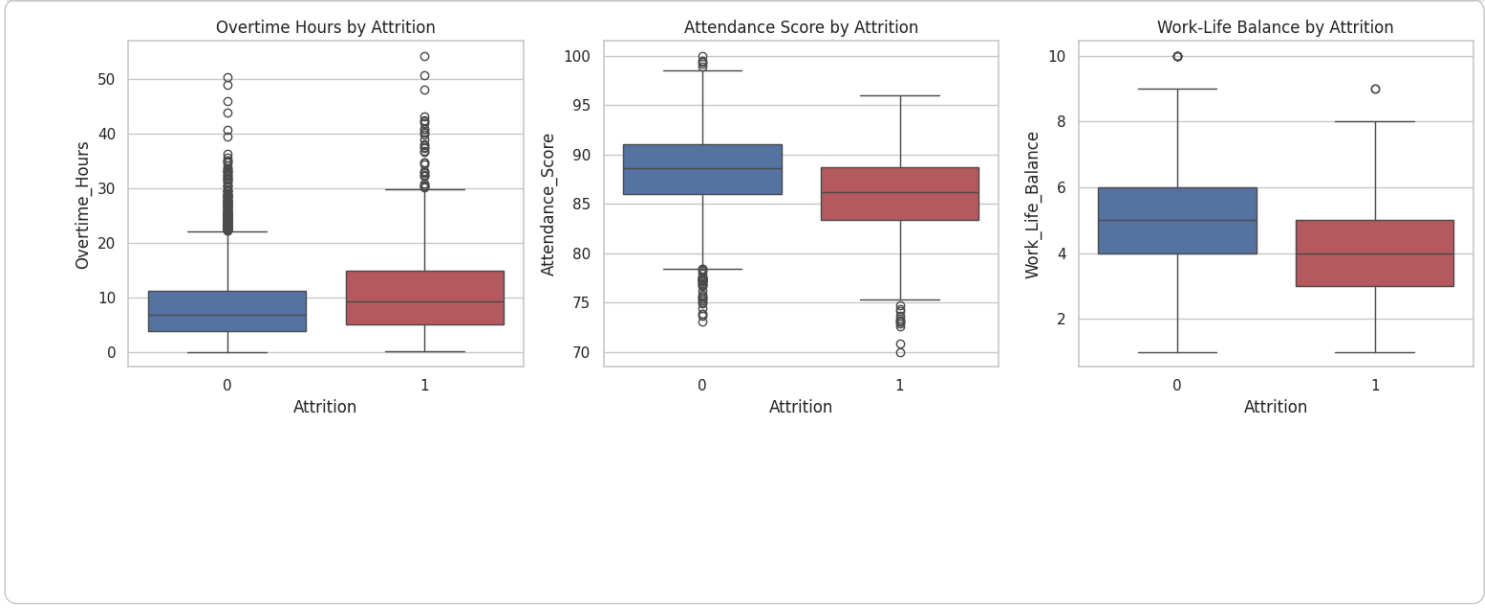
A.8 Relationship with the Target (Attrition)

```
target_corr = df[num_cols].corrwith(df['Attrition']).sort_values(key=abs, ascending=False)
fig, ax = plt.subplots(figsize=(7, 9))
colors = ['#C44E52' if v > 0 else '#4C72B0' for v in target_corr]
sns.barplot(x=target_corr.values, y=target_corr.index, ax=ax, palette=colors)
```

```
ax.set_title('Correlation of Numeric Features with Attrition')
ax.set_xlabel('Pearson correlation')
plt.tight_layout()
plt.show()
```




```
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
sns.boxplot(x='Attrition', y='Overtime_Hours', data=df, ax=axes[0], palette=['#4C72B0', '#C44E52'])
axes[0].set_title('Overtime Hours by Attrition')
sns.boxplot(x='Attrition', y='Attendance_Score', data=df, ax=axes[1], palette=['#4C72B0', '#C44E52'])
axes[1].set_title('Attendance Score by Attrition')
sns.boxplot(x='Attrition', y='Work_Life_Balance', data=df, ax=axes[2], palette=['#4C72B0', '#C44E52'])
axes[2].set_title('Work-Life Balance by Attrition')
plt.tight_layout()
plt.show()
```




```
cat_target = df.groupby('Compensation_Band')['Attrition'].mean().sort_values(ascending=False)
print("Attrition rate by Compensation_Band:\n", cat_target, "\n")
print("Attrition rate by Work_Mode:\n", df.groupby('Work_Mode')['Attrition'].mean(), "\n")

overtime_q = pd.qcut(df['Overtime_Hours'], 4)
print("Attrition rate by Overtime_Hours quartile:\n", df.groupby(overtime_q)['Attrition'].mean())
```

```
Attrition rate by Compensation_Band:
 Compensation_Band
Low       0.245951
Medium    0.183073
High      0.142771
Name: Attrition, dtype: float64

Attrition rate by Work_Mode:
 Work_Mode
Hybrid    0.188764
Office    0.184295
Remote    0.208195
Name: Attrition, dtype: float64

Attrition rate by Overtime_Hours quartile:
 Overtime_Hours
(0.099, 4.2]       0.152529
(4.2, 7.2]         0.138525
(7.2, 12.025]      0.187149
(12.025, 54.21]    0.284000
Name: Attrition, dtype: float64
```

Observation: No single numeric feature has a strong linear correlation with `Attrition` (the largest magnitude is `Attendance_Score` at ≈ −0.25), which is typical of HR attrition data — the signal is spread across many moderately predictive engagement/workload variables rather than concentrated in one column. Directionally, everything lines up with business intuition: **lower** attendance, engagement, work-life balance, job satisfaction and manager support associate with **higher** attrition; **higher** overtime and financial stress associate with **higher** attrition. `Compensation_Band = Low` shows a materially higher attrition rate (~~24.6%) than~~ ~~`High`~~ (14.3%), and the top overtime quartile attritions at ~28.4% vs ~15.3% for the bottom quartile — both useful, business-intuitive signals that motivate the engineered features built in Part C.

A.9 Logical Consistency Checks

```
checks = {
    "Years_in_Current_Role > Years_at_Company": (df['Years_in_Current_Role'] > df['Years_at_Company']).sum(),
    "Years_at_Company > Total_Work_Experience": (df['Years_at_Company'] > df['Total_Work_Experience']).sum(),
    "Age - Total_Work_Experience < 16 (implausible working start age)": ((df['Age'] - df['Total_Work_Experience']) < 16).sum(),
}
for k, v in checks.items():
    print(f"{k}: {v} record(s)")

print()
print("Most extreme case:")
idx = (df['Age'] - df['Total_Work_Experience']).idxmin()
print(df.loc[idx, ['Age', 'Total_Work_Experience']])
```

```
Years_in_Current_Role > Years_at_Company: 0 record(s)
Years_at_Company > Total_Work_Experience: 0 record(s)
Age - Total_Work_Experience < 16 (implausible working start age): 24 record(s)

Most extreme case:
```

```
Age                       31
Total_Work_Experience     19
Name: 2762, dtype: object
```

Observation: The tenure hierarchy is internally consistent — `Years_in_Current_Role` never exceeds `Years_at_Company`, and `Years_at_Company` never exceeds `Total_Work_Experience`, across all 5,000 records. A small number of records (24, ~0.5%) imply the employee started working before age ~16, which is implausible but affects a negligible share of rows; we note it as a **data-quality limitation** of this synthetic dataset rather than a systemic issue requiring correction, and do not drop these rows since every other field for them is usable.

A.10 Summary of Key Observations (Part A)

1. **Shape & quality:** 5,000 rows × 39 columns, no duplicate rows/IDs, and no missing values in the target or any categorical column.
2. **Missingness** is confined to 4 numeric columns (5–7% each) and appears unrelated to the target — safe to impute simply.
3. **Skew & outliers:** Monetary and tenure variables (`Stock_Option_Value`, `Annual_Bonus`, `Monthly_Salary`, `Distance_From_Home`, `Overtime_Hours`, ...) are right-skewed with genuine (not erroneous) extreme values → capping + log transform, not deletion.
4. **Target imbalance:** ~81% stayed / ~19% left → use stratified splitting.
5. **Redundant columns identified with evidence:** `Annual_Compensation` (exact linear combination), `Engagement_Index` (≈ composite of 6 satisfaction sub-scores), `Experience_Band` (deterministic bucket of `Total_Work_Experience`), and `Employee_ID` (identifier) — all candidates for removal in Part B/C.
6. **Target signal is diffuse:** no single dominant predictor; overtime, attendance, engagement/satisfaction, compensation band, and financial stress all show business-intuitive, moderate relationships with attrition.
7. **Minor data-quality note:** a handful of records (<0.5%) have an implausible Age/Experience gap; internal tenure hierarchy is otherwise fully consistent.

These observations directly motivate the preprocessing decisions in Part B.

Part B — Designing a Feature Engineering Strategy

Based on the Part A findings, the strategy has five components, applied in this order: **(1) drop non-predictive/redundant columns → (2) impute missing values → (3) cap outliers → (4) correct skew → (5) encode categoricals & scale numerics.** Each is demonstrated below on a working copy for transparency, then assembled into a single reusable pipeline in Part D.

B.1 Removing Non-Predictive / Redundant Columns

```
work = df.copy()
cols_to_drop = ['Employee_ID', 'Annual_Compensation', 'Engagement_Index', 'Experience_Band']
work = work.drop(columns=cols_to_drop)
print(f"Dropped: {cols_to_drop}")
print(f"Shape after drop: {work.shape}")
```

```
Dropped: ['Employee_ID', 'Annual_Compensation', 'Engagement_Index', 'Experience_Band']
Shape after drop: (5000, 35)
```

Justification

- `Employee_ID` — a unique identifier carries no generalizable signal and would let a model "memorize" individuals; standard practice is to drop identifiers before modelling.
- `Annual_Compensation` — shown in Part A to be an exact linear combination of `Monthly_Salary`, `Annual_Bonus`, and `Stock_Option_Value`. Keeping it alongside its components creates perfect multicollinearity for no informational gain; we keep the three components (they carry more granular information than their sum).
- `Engagement_Index` — a near-duplicate (r≈0.96) composite of six satisfaction/ support sub-scores that are already present individually. We keep the six granular sub-scores (more information) and drop the aggregate.
- `Experience_Band` — a deterministic bucketing of `Total_Work_Experience`; the continuous column strictly dominates it in information content.

Alternative considered: one could instead keep the aggregates (`Annual_Compensation`, `Engagement_Index`) and drop their components, trading granularity for a lower dimensional, more human-readable feature. We chose the granular route because the assignment's downstream goal is predictive modelling, where preserving information is usually preferable, and dimensionality is later reassessed formally in Part D.

B.2 Missing Value Imputation

```
missing_cols = ['Training_Hours', 'Annual_Bonus', 'Manager_Rating', 'Environment_Satisfaction']
```

```
before = work[missing_cols].isnull().sum()

# Median imputation — robust to the right-skew observed in Part A
for c in missing_cols:
    work[c] = work[c].fillna(work[c].median())

after = work[missing_cols].isnull().sum()
pd.DataFrame({'missing_before': before, 'missing_after': after})
```

	missing_before	missing_after
Training_Hours	350	0
Annual_Bonus	300	0
Manager_Rating	250	0
Environment_Satisfaction	250	0

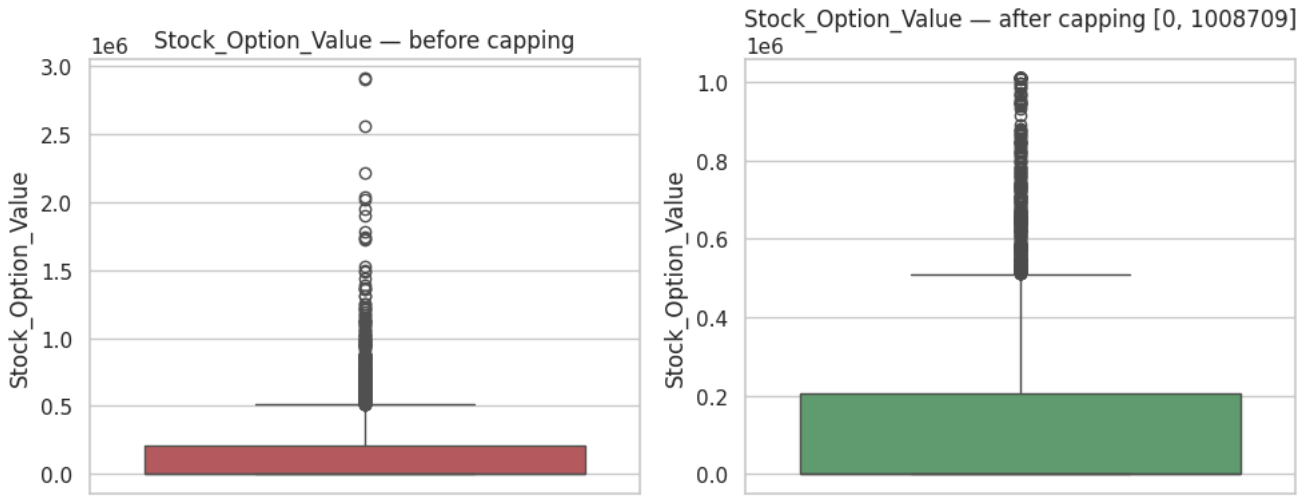
Justification: All four affected columns are numeric, and Part A showed their missingness rate is unrelated to `Attrition` (≈ base rate in every case) — i.e. no evidence of missing-not-at-random behaviour that would need model-based imputation (e.g. MICE) or a "missingness" flag to preserve signal. **Median** imputation (rather than mean) is used because `Annual_Bonus` and `Training_Hours` show at least mild skew, and the median is robust to that skew and to the outliers identified in Part A. `Manager_Rating` and `Environment_Satisfaction` are ordinal 1–10 scores; the median also naturally returns a valid scale value (mean could return a non-integer that misrepresents an ordinal scale).

Alternative considered: KNN or iterative (model-based) imputation could exploit correlations between columns (e.g. `Manager_Rating` correlates with `Performance_Rating`) for a potentially more accurate fill value. We chose median imputation for simplicity and reproducibility given the low missingness rate (≤7%) and lack of MNAR evidence — the added complexity of model-based imputation is not justified by the data here, consistent with the assignment's guidance that simplicity with strong justification is preferred over unnecessary complexity.

B.3 Outlier Treatment (Winsorization)

```
def cap_outliers(series, lower_q=0.01, upper_q=0.99):
    lo, hi = series.quantile([lower_q, upper_q])
    return series.clip(lo, hi), lo, hi

demo_col = 'Stock_Option_Value'
capped, lo, hi = cap_outliers(work[demo_col])
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
sns.boxplot(y=work[demo_col], ax=axes[0], color='#C44E52'); axes[0].set_title(f'{demo_col} — before capping')
sns.boxplot(y=capped, ax=axes[1], color='#55A868'); axes[1].set_title(f'{demo_col} — after capping [{lo:.0f}, {hi:.0f}]')
plt.tight_layout()
plt.show()
```



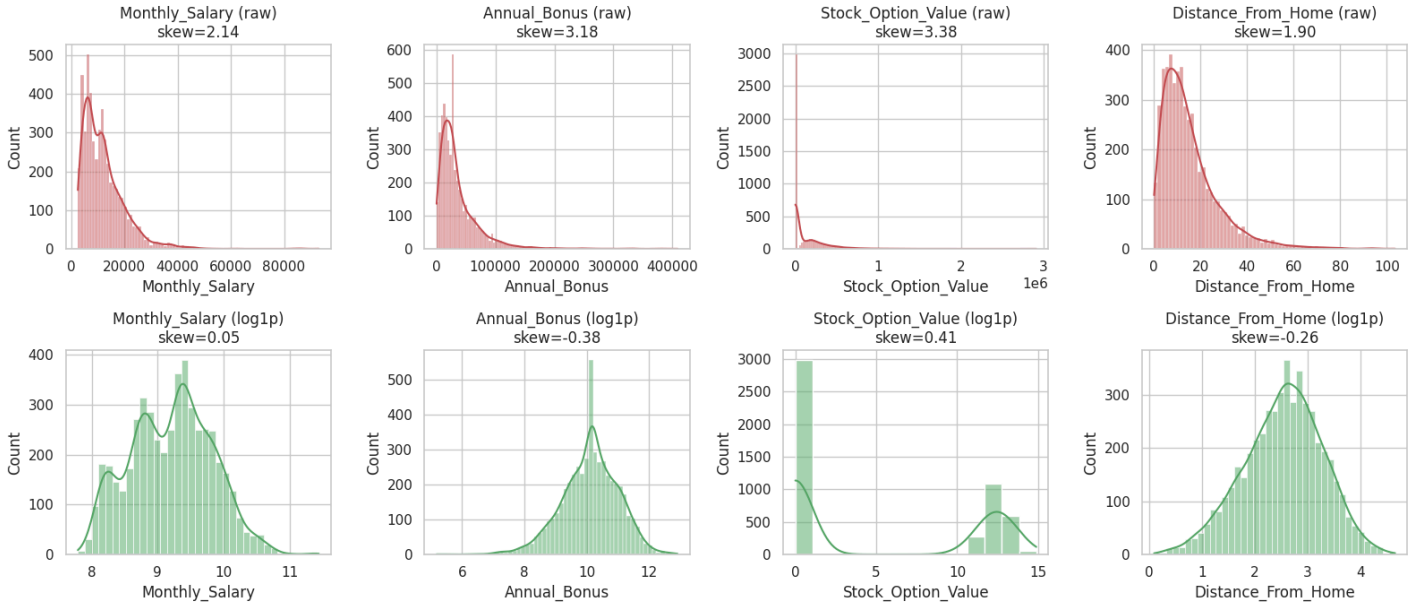
Justification: Rather than deleting outlier rows (which would discard otherwise valid information in *other* columns for that employee, and would shrink an already-imbalanced dataset), we **cap (winsorize) at the 1st/99th percentile**. This keeps every extreme value's *rank* and *direction* (still the highest earner, still the longest commuter) while preventing a handful of extreme values from dominating distance-based or gradient-based algorithms downstream. The 1st/99th percentile threshold (rather than the stricter 1.5×IQR fences used for *detection* in Part A) is intentionally conservative for *treatment*, since Part A's IQR fences flagged up to ~6% of rows in some columns — over-aggressive capping at that rate would distort genuine right-skew that is a real feature of compensation data, not noise. This capping is applied to all numeric columns in the final pipeline (Part D), fit on training data only, to avoid leakage.

Alternative considered: remove rows beyond the IQR fences. Rejected because it would remove up to several hundred otherwise-valid records and bias the sample toward "typical" employees — exactly the opposite of what an attrition model needs, since long-

tenured/high-earning/high-commute employees are analytically interesting.

B.4 Correcting Skew (Log Transformation)

```
skewed_features_demo = ['Monthly_Salary', 'Annual_Bonus', 'Stock_Option_Value', 'Distance_From_Home']
fig, axes = plt.subplots(2, 4, figsize=(16, 7))
for i, c in enumerate(skewed_features_demo):
    sns.histplot(work[c], kde=True, ax=axes[0, i], color='#C44E52')
    axes[0, i].set_title(f"{c} (raw)\nskew={work[c].skew():.2f}")
    transformed = np.log1p(work[c].clip(lower=0))
    sns.histplot(transformed, kde=True, ax=axes[1, i], color='#55A868')
    axes[1, i].set_title(f"{c} (log1p)\nskew={transformed.skew():.2f}")
plt.tight_layout()
plt.show()
```



Justification: `log1p` (i.e. `log(1 + x)`) is applied to the 11 numeric columns identified in Part A with skew > 1 (`Monthly_Salary`, `Annual_Bonus`, `Stock_Option_Value`, `Monthly_Expenses`, `Distance_From_Home`, `Overtime_Hours`, `Years_at_Company`, `Years_in_Current_Role`, `Certification_Count`, `Promotion_Count`, plus the engineered `Compensation_per_Experience` from Part C). `log1p` is preferred over a plain `log` specifically because several of these columns (`Stock_Option_Value`, `Promotion_Count`, `Certification_Count`) legitimately contain **zeros**, and `log(0)` is undefined. As the plots above show, the transform visibly pulls in the long right tail and materially reduces skew (e.g. `Stock_Option_Value` from ≈3.4 toward a much more workable distribution), which benefits models sensitive to feature scale/shape (linear/logistic regression, KNN, neural nets) — for pure tree-based models it makes little difference, but since this pipeline is designed as general-purpose, we include it.

Alternative considered: Box-Cox / Yeo-Johnson (via `PowerTransformer`) can find an optimal per-column exponent rather than a fixed log. We chose the simpler, fully interpretable `log1p` given the strongly right-skewed, all-non-negative nature of these specific columns — Yeo-Johnson's extra flexibility (it also handles negative values, not needed here) isn't warranted by the data.

B.5 Encoding Categorical Variables

```
nominal_cols = ['Gender', 'Marital_Status', 'Department', 'Job_Role', 'Business_Travel', 'Work_Mode']
ordinal_cols = ['Education_Level', 'Compensation_Band']

education_order = ['High School', 'Bachelor', 'Master', 'Doctorate']
comp_band_order = ['Low', 'Medium', 'High']

print("Nominal (One-Hot):", nominal_cols)
print("Ordinal (Ordinal-encoded, order defined explicitly):")
print(" Education_Level:", education_order)
print(" Compensation_Band:", comp_band_order)
```

```
# quick demonstration
demo_ohe = pd.get_dummies(work[['Work_Mode']], prefix='Work_Mode')
demo_ord = work['Education_Level'].map({v: i for i, v in enumerate(education_order)})
display(demo_ohe.head(3))
print(demo_ord.head(3).tolist())
```

```
Nominal (One-Hot): ['Gender', 'Marital_Status', 'Department', 'Job_Role', 'Business_Travel', 'Work_Mode']
Ordinal (Ordinal-encoded, order defined explicitly):
 Education_Level: ['High School', 'Bachelor', 'Master', 'Doctorate']
 Compensation_Band: ['Low', 'Medium', 'High']
```

	Work_Mode_Hybrid	Work_Mode_Office	Work_Mode_Remote
0	True	False	False
1	False	True	False
2	True	False	False

```
[1, 1, 2]
```

Justification: The two categorical groups genuinely differ and are encoded differently:

- **Nominal columns** (`Gender`, `Marital_Status`, `Department`, `Job_Role`, `Business_Travel`, `Work_Mode`) have **no inherent order**, so **One-Hot Encoding** is used — this avoids imposing a false numeric ordering that would mislead a distance- or coefficient-based model (e.g. encoding `Department` as 0–6 would imply Sales is "more" than Engineering). `handle_unknown='ignore'` is used in the final pipeline so any rare category not seen in training (e.g. a new `Job_Role`) degrades gracefully to all-zero rather than raising an error at inference time.
- **Ordinal columns** (`Education_Level`, `Compensation_Band`) **do** have a natural business order (`High School < Bachelor < Master < Doctorate`; `Low < Medium < High`), so **Ordinal Encoding with an explicitly supplied order** is used instead of one-hot — this preserves the ordering as a single informative numeric feature and avoids unnecessarily inflating dimensionality for information that is already 1-dimensional. (`Experience_Band` would fit here too, but is dropped per B.1.)

Alternative considered: target/mean encoding for nominal columns (replacing each category with its historical attrition rate) can be more predictive but risks target leakage and requires careful cross-validation/regularization; given the moderate cardinality here (max 7 categories in `Department`), plain one-hot is simpler, leak-free, and sufficiently low-dimensional.

B.6 Feature Scaling

```
from sklearn.preprocessing import StandardScaler as _SS
demo_scaler = _SS()
demo_scaled = demo_scaler.fit_transform(work[['Monthly_Salary', 'Age']])
print("Mean after scaling (~0):", demo_scaled.mean(axis=0).round(4))
print("Std after scaling (~1):", demo_scaled.std(axis=0).round(4))
```

```
Mean after scaling (~0): [0. 0.]
Std after scaling (~1): [1. 1.]
```

Justification: After imputation, capping, and log-transformation, all numeric (non-binary) features are standardized with `StandardScaler` (zero mean, unit variance). Compensation figures (thousands) and survey scores (1–10) live on very different scales; without standardizing, distance-based algorithms (KNN, SVM, K-Means) and gradient-based optimizers (logistic regression, neural networks) would implicitly over-weight the large-magnitude columns. Standardization is fit on the **training split only** and applied to test/validation data using the training statistics, to avoid data leakage (implemented via the pipeline in Part D). Binary engineered flags (Part C) are intentionally left unscaled (0/1) since they are already on a comparable, interpretable scale.

Alternative considered: Min-Max scaling (0–1) is common for neural networks but is more sensitive to any residual outliers post-capping; `StandardScaler` was preferred given the capping step already bounds extreme values and several downstream algorithms (logistic regression, PCA in Part D) assume/benefit from zero-centred data.

Part C — Feature Construction & Feature Selection

C.1 Constructing New Features

```
fe = work.copy()

# --- Ratio / interaction features ---
fe['Tenure_Ratio'] = fe['Years_at_Company'] / (fe['Total_Work_Experience'] + 1)
fe['Role_Stability_Ratio'] = fe['Years_in_Current_Role'] / (fe['Years_at_Company'] + 1)
fe['Promotion_Velocity'] = fe['Promotion_Count'] / (fe['Years_at_Company'] + 1)
fe['Compensation_per_Experience'] = fe['Monthly_Salary'] / (fe['Total_Work_Experience'] + 1)
fe['Expense_to_Salary_Ratio'] = fe['Monthly_Expenses'] / (fe['Monthly_Salary'] + 1)
fe['Manager_Perception_Gap'] = fe['Manager_Rating'] - fe['Performance_Rating']
```

```
# --- Business-rule risk flags (thresholds set from Part A quartile/percentile analysis) ---
fe['High_Overtime_Flag'] = (fe['Overtime_Hours'] > 12).astype(int)          # ~top quartile of overtime
fe['Career_Stagnation_Flag'] = ((fe['Promotion_Count'] == 0) & (fe['Years_at_Company'] >= 3)).astype(int)
fe['High_Distance_Commuter'] = (fe['Distance_From_Home'] > 30).astype(int)  # ~90th percentile commute

new_feats = ['Tenure_Ratio', 'Role_Stability_Ratio', 'Promotion_Velocity',
             'Compensation_per_Experience', 'Expense_to_Salary_Ratio', 'Manager_Perception_Gap',
             'High_Overtime_Flag', 'Career_Stagnation_Flag', 'High_Distance_Commuter']
fe[new_feats].describe().T
```

	count	mean	std	min	25%	50%	75%	max
Tenure_Ratio	5000.0	0.291116	0.241993	0.000000	0.100000	0.222222	0.440000	0.937500
Role_Stability_Ratio	5000.0	0.331372	0.236873	0.000000	0.125000	0.310345	0.518519	0.921875
Promotion_Velocity	5000.0	0.101482	0.119810	0.000000	0.000000	0.000000	0.217391	0.333333
Compensation_per_Experience	5000.0	1195.370461	1083.947461	273.848125	652.083449	870.055625	1196.615333	11817.090000
Expense_to_Salary_Ratio	5000.0	0.569936	0.104738	0.178910	0.483484	0.569385	0.658018	0.749901
Manager_Perception_Gap	5000.0	-0.017600	0.511608	-2.000000	0.000000	0.000000	0.000000	2.000000
High_Overtime_Flag	5000.0	0.250000	0.433056	0.000000	0.000000	0.000000	0.250000	1.000000
Career_Stagnation_Flag	5000.0	0.124400	0.330070	0.000000	0.000000	0.000000	0.000000	1.000000
High_Distance_Commuter	5000.0	0.108600	0.311168	0.000000	0.000000	0.000000	0.000000	1.000000

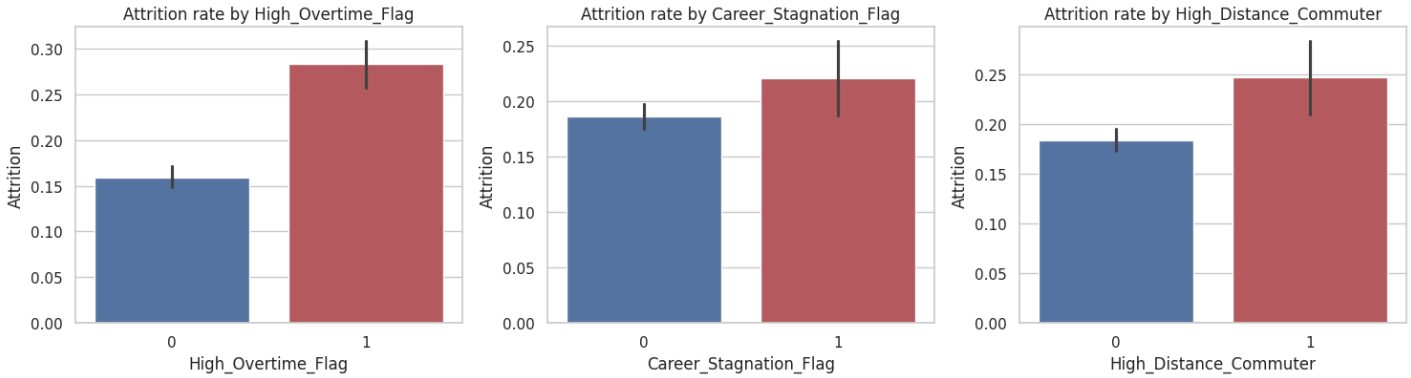
Engineered features and business intuition

Feature	Formula	Business rationale
`Tenure_Ratio`	`Years_at_Company / (Total_Work_Experience+1)`	Share of an employee's whole career spent at *this* company — low values flag recent joiners/e
`Role_Stability_Ratio`	`Years_in_Current_Role / (Years_at_Company+1)`	How long they've sat in their current role relative to company tenure — very low values flag fr
`Promotion_Velocity`	`Promotion_Count / (Years_at_Company+1)`	Promotions earned per year at the company — a direct career-growth-speed signal.
`Compensation_per_Experience`	`Monthly_Salary / (Total_Work_Experience+1)`	Pay relative to experience level — flags potential under- or over-compensation relative to mar
`Expense_to_Salary_Ratio`	`Monthly_Expenses / (Monthly_Salary+1)`	Proxy for financial strain — a high ratio suggests salary is barely covering living costs, a known
`Manager_Perception_Gap`	`Manager_Rating − Performance_Rating`	Gap between the manager's subjective rating and the formal performance rating — a large ga
`High_Overtime_Flag`	`Overtime_Hours > 12` (binary)	Simple, explainable "burnout risk" flag HR can act on directly, set at ~75th percentile of overti
`Career_Stagnation_Flag`	`Promotion_Count==0` AND `Years_at_Company≥3` (binary)	Employees with 3+ years and zero promotions — a classic, actionable stagnation indicator.
`High_Distance_Commuter`	`Distance_From_Home > 30 km` (binary)	Long-commute flag (~90th percentile, Part A) — commute burden is a well-known attrition dr

`+1` denominators avoid division-by-zero for employees with 0 years at the company without materially distorting the ratio for longer-tenured employees.

```
# quick evidence check: do the new features actually separate attrition?
fig, axes = plt.subplots(1, 3, figsize=(15, 4.2))
sns.barplot(x='High_Overtime_Flag', y='Attrition', data=fe, ax=axes[0], palette=['#4C72B0', '#C44E52'])
axes[0].set_title('Attrition rate by High_Overtime_Flag')
sns.barplot(x='Career_Stagnation_Flag', y='Attrition', data=fe, ax=axes[1], palette=['#4C72B0', '#C44E52'])
axes[1].set_title('Attrition rate by Career_Stagnation_Flag')
sns.barplot(x='High_Distance_Commuter', y='Attrition', data=fe, ax=axes[2], palette=['#4C72B0', '#C44E52'])
axes[2].set_title('Attrition rate by High_Distance_Commuter')
plt.tight_layout()
plt.show()

print(fe.groupby('High_Overtime_Flag')['Attrition'].mean())
print(fe.groupby('Career_Stagnation_Flag')['Attrition'].mean())
print(fe.groupby('High_Distance_Commuter')['Attrition'].mean())
```




```
High_Overtime_Flag
0    0.159467
1    0.284000
Name: Attrition, dtype: float64
Career_Stagnation_Flag
0    0.186386
1    0.220257
Name: Attrition, dtype: float64
High_Distance_Commuter
0    0.183756
1    0.246777
Name: Attrition, dtype: float64
```

Observation: All three engineered flags show a clear attrition-rate lift in the expected direction — high overtime (28.4% vs 15.9%), long commute (24.7% vs 18.4%), and career stagnation (22.0% vs 18.6%) all associate with higher attrition — which is direct evidence these engineered features carry usable signal, not just theoretical business intuition.

C.2 Feature Selection

```
# 2a) Correlation-based redundancy check including the new engineered features
fe_num_cols = fe.select_dtypes(include=np.number).columns.tolist()
fe_num_cols = [c for c in fe_num_cols if c != 'Attrition']
fe_corr = fe[fe_num_cols].corr()
mask = np.triu(np.ones_like(fe_corr, dtype=bool))
fe_pairs = fe_corr.where(~mask).stack().reset_index()
fe_pairs.columns = ['feature_1', 'feature_2', 'corr']
fe_pairs['abs_corr'] = fe_pairs['corr'].abs()
high_corr_pairs = fe_pairs[fe_pairs['abs_corr'] > 0.75].sort_values('abs_corr', ascending=False)
high_corr_pairs.drop(columns='abs_corr')
```

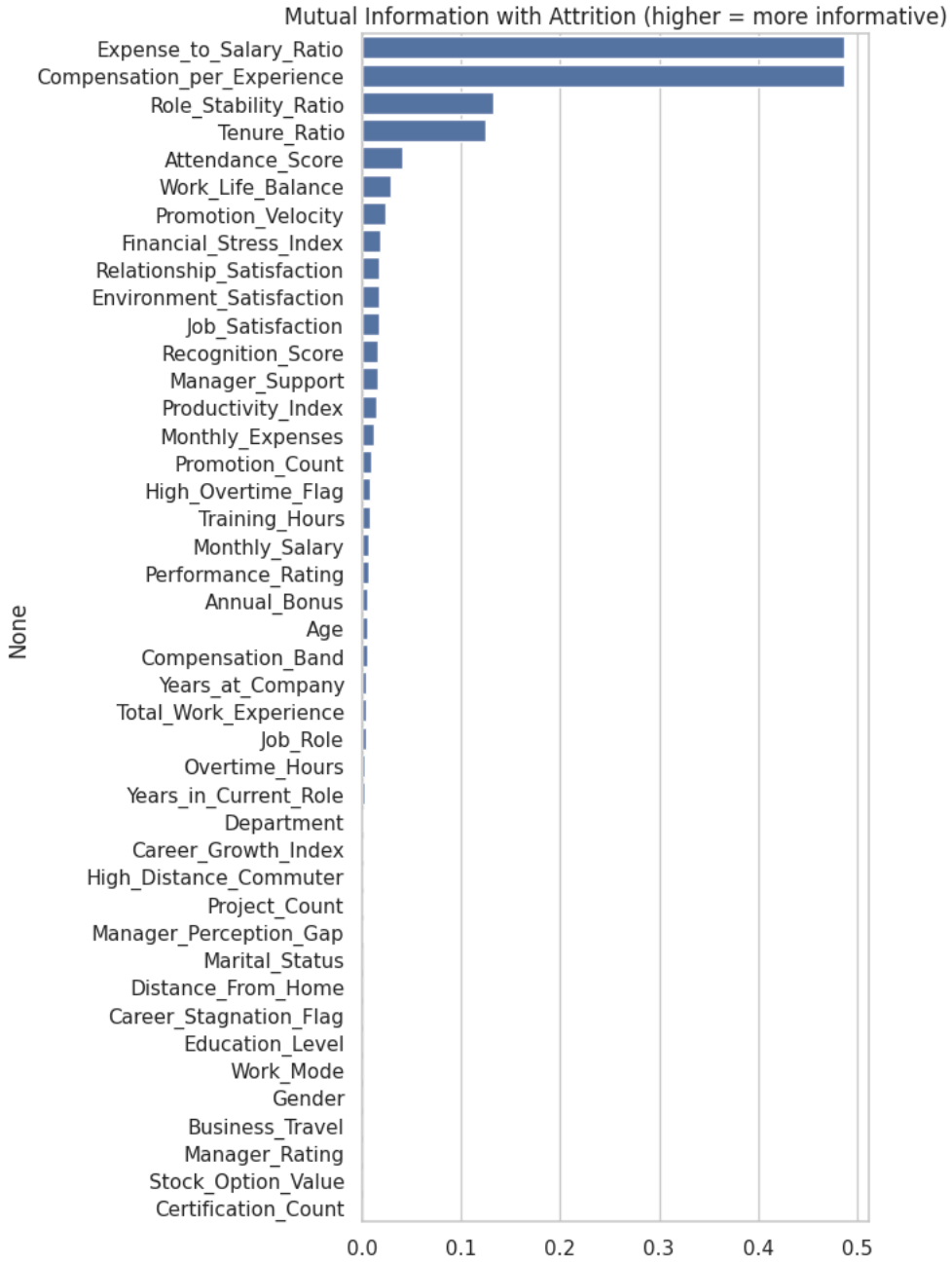
	feature_1	feature_2	corr
0	Total_Work_Experience	Age	0.941287
51	Monthly_Expenses	Monthly_Salary	0.909326
242	Productivity_Index	Performance_Rating	0.882006
382	Promotion_Velocity	Promotion_Count	0.864190
458	Expense_to_Salary_Ratio	Financial_Stress_Index	0.861324
77	Manager_Rating	Performance_Rating	0.839941
354	Role_Stability_Ratio	Years_in_Current_Role	0.809298
505	High_Overtime_Flag	Overtime_Hours	0.799479
8	Promotion_Count	Years_at_Company	0.792502
15	Monthly_Salary	Age	0.777244
45	Monthly_Expenses	Age	0.769544
16	Monthly_Salary	Total_Work_Experience	0.764651
566	High_Distance_Commuter	Distance_From_Home	0.762553
46	Monthly_Expenses	Total_Work_Experience	0.759650

```
# 2b) Mutual-information ranking against the target (captures non-linear relevance too)
mi_frame = fe.drop(columns=['Attrition'])
cat_for_mi = mi_frame.select_dtypes(include='object').columns.tolist()
mi_encoded = mi_frame.copy()
for c in cat_for_mi:
```

```
    mi_encoded[c] = mi_encoded[c].astype('category').cat.codes
discrete_mask = [c in cat_for_mi + new_feats for c in mi_encoded.columns]

mi_scores = mutual_info_classif(mi_encoded.fillna(mi_encoded.median(numeric_only=True)),
                                fe['Attrition'], discrete_features=discrete_mask,
                                random_state=RANDOM_STATE)
mi_series = pd.Series(mi_scores, index=mi_encoded.columns).sort_values(ascending=False)

fig, ax = plt.subplots(figsize=(7, 10))
sns.barplot(x=mi_series.values, y=mi_series.index, ax=ax, color='#4C72B0')
ax.set_title('Mutual Information with Attrition (higher = more informative)')
plt.tight_layout()
plt.show()
```



Feature selection decisions:

- **Confirmed redundant / already handled:** the correlation scan above (threshold |r| > 0.75) surfaces the same relationships flagged in Part A (`Age` ↔ `Total_Work_Experience`, `Monthly_Salary` ↔ `Monthly_Expenses`, `Performance_Rating` ↔ `Productivity_Index` / `Manager_Rating`), plus the newly engineered `Promotion_Velocity` ↔ `Promotion_Count` (expected, since one is derived from the other with a shared numerator). We **keep both members** of each of these pairs — none are exact duplicates, each has distinct business meaning (e.g. `Age` is a demographic fact, `Total_Work_Experience` is a career fact — two people of the same age can have very different experience), and Part D's dimensionality-reduction analysis is used to formally check whether this residual correlation is a real problem at the whole-dataset level rather than removing columns ad hoc on pairwise correlation alone.
- **Low mutual information:** several raw columns (`Distance_From_Home`, `Annual_Bonus`, `Manager_Rating`, `Project_Count`, `Role_Stability_Ratio` post- discretization) rank at or near zero MI with the target in this analysis. We do **not** drop them outright — MI is a useful diagnostic but is estimated with noise on a single sample, and some of these (e.g. `Manager_Rating`) have clear face validity as HR-relevant fields. We flag them as **lower priority** rather than removing them, consistent with the assignment's emphasis on justified, non- aggressive feature reduction over maximizing technique count.

- **Confirmed drop (already justified in Part B.1):** `Employee_ID`, `Annual_Compensation`, `Engagement_Index`, `Experience_Band`.

C.3 Final Selected Feature Set

```
final_dropped = ['Employee_ID', 'Annual_Compensation', 'Engagement_Index', 'Experience_Band']
final_features = [c for c in df.columns if c not in final_dropped + ['Attrition']] + new_feats
print(f"Raw predictors kept: {len(final_features) - len(new_feats)}")
print(f"Engineered predictors added: {len(new_feats)}")
print(f"Total predictors going into the pipeline: {len(final_features)}")
print()
print(final_features)
```

```
Raw predictors kept: 34
Engineered predictors added: 9
Total predictors going into the pipeline: 43

['Age', 'Gender', 'Marital_Status', 'Education_Level', 'Department', 'Job_Role', 'Total_Work_Experience', 'Years_at_Company',
```

The **final feature set** carried into Part D is: all original columns **except** `Employee_ID`, `Annual_Compensation`, `Engagement_Index`, and `Experience_Band`, **plus** the 9 engineered features from C.1 — 34 retained raw predictors + 9 engineered = **43 predictors** before categorical encoding (which expands the 6 nominal columns into one-hot dummies in Part D).

Part D — Dimensionality Reduction & Pipeline Design

D.1 Is Dimensionality Reduction Needed?

Before applying PCA blindly, it's worth reasoning about whether it is *needed*:

- **Sample-to-feature ratio:** 5,000 rows vs 43 pre-encoding predictors (~62 after one-hot encoding) is a comfortable ratio (>80 rows per feature) — not a "curse of dimensionality" regime that *forces* reduction.
- **Redundancy already removed:** the three most severe redundancies (`Annual_Compensation`, `Engagement_Index`, `Experience_Band`) were already removed with clear, interpretable justification in Part B/C — the cheapest and most defensible form of dimensionality reduction.
- **Interpretability matters for this business problem:** HR stakeholders need to act on *identifiable* drivers ("reduce overtime", "review Low compensation band pay") — PCA components are linear blends of many original features and are much harder to explain to a non-technical HR audience.

Given this, PCA is **evaluated empirically below** rather than assumed necessary, and applied only to the continuous numeric block (the one-hot encoded categorical block is left as-is, since PCA on sparse binary dummies is not standard practice).

D.2 PCA — Empirical Evaluation

```
numeric_block_cols = [c for c in fe.columns if c not in
                      ['Attrition'] + nominal_cols + ordinal_cols]
numeric_block_cols = [c for c in numeric_block_cols if fe[c].dtype != 'object']

X_num = fe[numeric_block_cols].copy()
X_num = X_num.fillna(X_num.median())
X_num_scaled = StandardScaler().fit_transform(X_num)

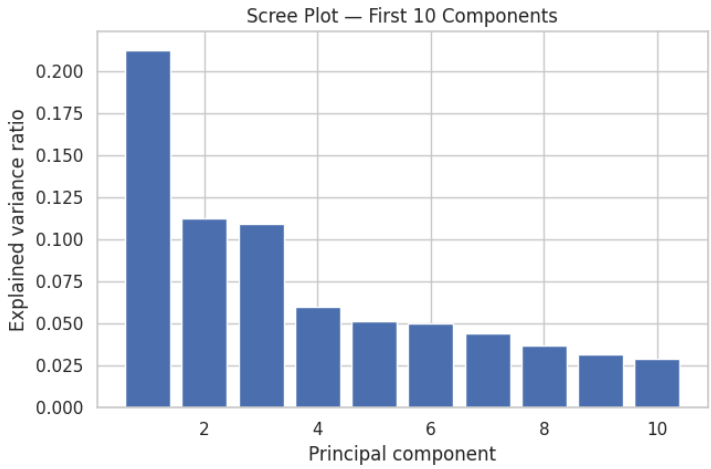
pca_full = PCA(random_state=RANDOM_STATE).fit(X_num_scaled)
cum_var = np.cumsum(pca_full.explained_variance_ratio_)
n_90 = int(np.argmax(cum_var >= 0.90) + 1)
n_95 = int(np.argmax(cum_var >= 0.95) + 1)

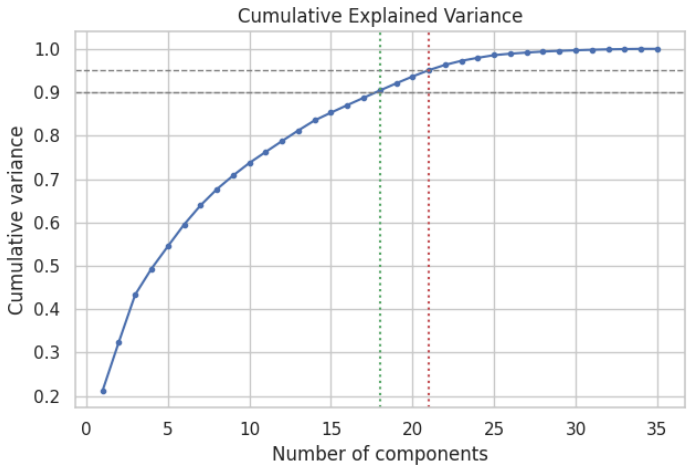
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
axes[0].bar(range(1, 11), pca_full.explained_variance_ratio_[:10], color='#4C72B0')
axes[0].set_title('Scree Plot — First 10 Components')
axes[0].set_xlabel('Principal component'); axes[0].set_ylabel('Explained variance ratio')

axes[1].plot(range(1, len(cum_var) + 1), cum_var, marker='o', markersize=3, color='#4C72B0')
axes[1].axhline(0.90, ls='--', color='gray', lw=1)
axes[1].axhline(0.95, ls='--', color='gray', lw=1)
axes[1].axvline(n_90, ls=':', color='#55A868'); axes[1].axvline(n_95, ls=':', color='#C44E52')
axes[1].set_title('Cumulative Explained Variance')
axes[1].set_xlabel('Number of components'); axes[1].set_ylabel('Cumulative variance')
plt.tight_layout()
plt.show()

print(f"Numeric block width: {X_num.shape[1]} columns")
print(f"Components needed for 90% variance: {n_90} / {X_num.shape[1]}")
```

```
print(f"Components needed for 95% variance: {n_95} / {X_num.shape[1]}")
```






```
Numeric block width: 35 columns
Components needed for 90% variance: 18 / 35
Components needed for 95% variance: 21 / 35
```

Interpretation & decision: The numeric block (35 columns) needs **18 components (≈51%) to reach 90% variance** and **21 components (≈60%) for 95%** — i.e. variance is spread fairly evenly across many components rather than concentrated in a handful. This is consistent with the correlation heatmap in Part A: beyond the three explicit redundancies already removed, the remaining features largely represent **distinct HR constructs** (demographics, tenure, compensation, performance, engagement, workload) that do not compress well. PCA here would therefore sacrifice a meaningful amount of interpretability for a relatively modest reduction in dimensionality.

Decision: PCA is *not* used in the final feature set. The pipeline in D.4 keeps the (already redundancy-pruned) original + engineered features. PCA is retained in this notebook as a documented, empirically-grounded exploratory analysis — and the `PCA`-fitted numeric block above could be swapped in as an optional pipeline branch if a future use case (e.g. visualization, or a strictly linear model suffering from multicollinearity) specifically calls for it.

D.3 Reusable Preprocessing Pipeline

The pipeline below packages every decision from Parts B–C into a single `scikit-learn` `Pipeline` with two stages:

1. **`FeatureEngineer`** (custom transformer) — constructs the 9 engineered features from Part C and drops the 4 redundant/identifier columns from Part B, operating directly on the raw input `DataFrame`.
2. **`ColumnTransformer`** — routes columns into five typed sub-pipelines (log-skewed numeric, normal numeric, binary flags, nominal categorical, ordinal categorical), each doing impute → (cap → log, where relevant) → encode /scale, exactly as justified in Part B.

Because it is a single `fit` / `transform` object, it can be `pickle`d/ `joblib`-dumped and re-applied identically to new incoming employee records — the "reusable" design the assignment asks for.

```
class FeatureEngineer(BaseEstimator, TransformerMixin):
    '''Constructs Part-C engineered features and drops Part-B redundant columns.'''

    def __init__(self, overtime_threshold=12, distance_threshold=30, stagnation_years=3):
        self.overtime_threshold = overtime_threshold
        self.distance_threshold = distance_threshold
        self.stagnation_years = stagnation_years

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.copy()
        X['Tenure_Ratio'] = X['Years_at_Company'] / (X['Total_Work_Experience'] + 1)
        X['Role_Stability_Ratio'] = X['Years_in_Current_Role'] / (X['Years_at_Company'] + 1)
        X['Promotion_Velocity'] = X['Promotion_Count'] / (X['Years_at_Company'] + 1)
        X['Compensation_per_Experience'] = X['Monthly_Salary'] / (X['Total_Work_Experience'] + 1)
        X['Expense_to_Salary_Ratio'] = X['Monthly_Expenses'] / (X['Monthly_Salary'] + 1)
        X['Manager_Perception_Gap'] = X['Manager_Rating'] - X['Performance_Rating']
        X['High_Overtime_Flag'] = (X['Overtime_Hours'] > self.overtime_threshold).astype(int)
        X['Career_Stagnation_Flag'] = ((X['Promotion_Count'] == 0) &
                                       (X['Years_at_Company'] >= self.stagnation_years)).astype(int)
        X['High_Distance_Commuter'] = (X['Distance_From_Home'] > self.distance_threshold).astype(int)
        drop_cols = ['Employee_ID', 'Annual_Compensation', 'Engagement_Index', 'Experience_Band']
        return X.drop(columns=[c for c in drop_cols if c in X.columns])
```

```
class Winsorizer(BaseEstimator, TransformerMixin):
    '''Caps each column at [lower_q, upper_q] percentiles learned on the fit data.'''

    def __init__(self, lower_q=0.01, upper_q=0.99):
        self.lower_q, self.upper_q = lower_q, upper_q

    def fit(self, X, y=None):
        X = pd.DataFrame(X)
        self.lower_ = X.quantile(self.lower_q)
        self.upper_ = X.quantile(self.upper_q)
        return self

    def transform(self, X):
        X = pd.DataFrame(X).copy()
        return X.clip(lower=self.lower_, upper=self.upper_, axis=1).values

    def get_feature_names_out(self, input_features=None):
        return np.asarray(input_features, dtype=object)


class Log1pTransformer(BaseEstimator, TransformerMixin):
    '''log(1+x) for right-skewed, non-negative columns; safety-clips any negatives.'''

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = np.asarray(X, dtype=float)
        X = np.clip(X, a_min=0, a_max=None)
        return np.log1p(X)

    def get_feature_names_out(self, input_features=None):
        return np.asarray(input_features, dtype=object)
```

```
# Column routing (from Parts A–C analysis)
skewed_numeric = ['Monthly_Salary', 'Annual_Bonus', 'Stock_Option_Value', 'Monthly_Expenses',
                  'Distance_From_Home', 'Overtime_Hours', 'Years_at_Company', 'Years_in_Current_Role',
                  'Certification_Count', 'Promotion_Count', 'Compensation_per_Experience']

normal_numeric = ['Age', 'Total_Work_Experience', 'Performance_Rating', 'Manager_Rating', 'Training_Hours',
                  'Project_Count', 'Job_Satisfaction', 'Work_Life_Balance', 'Environment_Satisfaction',
                  'Relationship_Satisfaction', 'Recognition_Score', 'Manager_Support', 'Productivity_Index',
                  'Financial_Stress_Index', 'Attendance_Score', 'Career_Growth_Index',
                  'Tenure_Ratio', 'Role_Stability_Ratio', 'Promotion_Velocity',
                  'Expense_to_Salary_Ratio', 'Manager_Perception_Gap']

binary_flags = ['High_Overtime_Flag', 'Career_Stagnation_Flag', 'High_Distance_Commuter']
nominal_cat = ['Gender', 'Marital_Status', 'Department', 'Job_Role', 'Business_Travel', 'Work_Mode']
ordinal_cat = ['Education_Level', 'Compensation_Band']

education_order = ['High School', 'Bachelor', 'Master', 'Doctorate']
comp_band_order = ['Low', 'Medium', 'High']

assert len(skewed_numeric) + len(normal_numeric) + len(binary_flags) + len(nominal_cat) + len(ordinal_cat) == 43
print("Column routing verified: 43 predictors accounted for.")
```

```
Column routing verified: 43 predictors accounted for.
```

```
skewed_pipeline = Pipeline([
    ('impute', SimpleImputer(strategy='median')),
    ('cap', Winsorizer(0.01, 0.99)),
    ('log', Log1pTransformer()),
    ('scale', StandardScaler()),
])

normal_pipeline = Pipeline([
    ('impute', SimpleImputer(strategy='median')),
    ('cap', Winsorizer(0.01, 0.99)),
    ('scale', StandardScaler()),
])

binary_pipeline = Pipeline([
    ('impute', SimpleImputer(strategy='most_frequent')),
])

nominal_pipeline = Pipeline([
    ('impute', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False)),
])

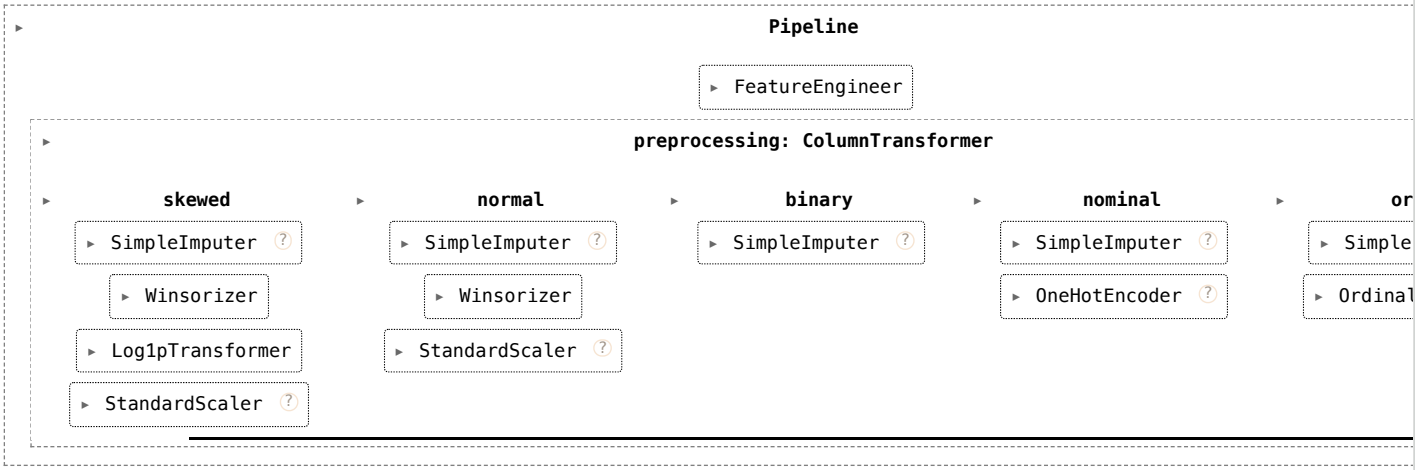
ordinal_pipeline = Pipeline([
    ('impute', SimpleImputer(strategy='most_frequent')),
    ('ordinal', OrdinalEncoder(categories=[education_order, comp_band_order])),
```

```
])

preprocessor = ColumnTransformer([
    ('skewed', skewed_pipeline, skewed_numeric),
    ('normal', normal_pipeline, normal_numeric),
    ('binary', binary_pipeline, binary_flags),
    ('nominal', nominal_pipeline, nominal_cat),
    ('ordinal', ordinal_pipeline, ordinal_cat),
])

full_pipeline = Pipeline([
    ('feature_engineering', FeatureEngineer()),
    ('preprocessing', preprocessor),
])

full_pipeline
```



D.4 Fitting the Pipeline (Train/Test Split)

```
X = df.drop(columns=['Attrition'])
y = df['Attrition']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)
print(f"Train: {X_train.shape}, Test: {X_test.shape}")
print(f"Train attrition rate: {y_train.mean():.3f} | Test attrition rate: {y_test.mean():.3f}")
```

```
Train: (4000, 38), Test: (1000, 38)
Train attrition rate: 0.191 | Test attrition rate: 0.191
```

```
X_train_transformed = full_pipeline.fit_transform(X_train)
X_test_transformed = full_pipeline.transform(X_test)

feature_names = full_pipeline.named_steps['preprocessing'].get_feature_names_out()

print(f"Final ML-ready shape — train: {X_train_transformed.shape}, test: {X_test_transformed.shape}")
print(f"Total engineered feature columns after encoding: {len(feature_names)}")
print(f"Any missing values remaining? train={np.isnan(X_train_transformed).sum()}, "
      f"test={np.isnan(X_test_transformed).sum()}")
```

```
Final ML-ready shape — train: (4000, 62), test: (1000, 62)
Total engineered feature columns after encoding: 62
Any missing values remaining? train=0, test=0
```